

Circuits VLSI pour l'IA

Conception d'un multiplieur de Booth 16×16
et intégration dans un accélérateur de convolution 2D

Aristide-Herve DOSSOU Prénom

Master 2 EEA – 2025/2026

Table des matières

1	Introduction	3
2	Multiplieur de Booth	3
2.1	Principe du codage de Booth	3
2.2	Table d'encodage Booth Radix-4	3
2.3	Architecture générale du multiplieur	3
2.3.1	Bloc Transformer	4
2.3.2	un encodeur Booth Radix-4	4
2.3.3	générateur de produits partiels basé sur un multiplexeur	4
2.3.4	un arbre d'addition de type CSA (Carry Save Adder)	5
2.3.5	un additionneur final CPA (Carry Propagate Adder)	7
3	Convolution 2D	7
3.1	Principe de la convolution	7
3.2	Noyaux de Sobel	8
3.3	Cœur de convolution	8
3.4	Module de stockage et de génération de fenêtre : Line Buffer	9
4	Module top : Chaîne complète de convolution 2D	11
4.1	Architecture fonctionnelle	11
5	Validation du multiplieur	12
6	Validation fonctionnelle de la convolution	12
6.1	Image d'entrée	12
6.2	Noyau de convolution utilisé	13
6.3	Calcul de la convolution : exemples détaillés	13
6.3.1	Exemple de calcul 1	13
6.3.2	Exemple de calcul 2	13
6.4	Résultat final de la convolution	14
6.5	Filtre de Sobel vertical	14
6.5.1	Noyau de Sobel vertical	14
6.5.2	Analyse théorique sur l'image de test	14
6.5.3	Exemple de calcul	15
6.5.4	Résultat global	15
7	Synthèse et placement-routage	15
8	Conclusion	18

1 Introduction

Les opérations arithmétiques de base, et en particulier l'addition et la multiplication, constituent le cœur des traitements de signal numérique tels que le filtrage, la convolution ou l'analyse d'images. Dans le contexte des accélérateurs matériels pour l'intelligence artificielle, ces opérations doivent être réalisées avec des contraintes fortes en termes de latence, de surface et de consommation.

L'objectif de ce projet est double. Dans un premier temps, il s'agit de concevoir un multiplieur signé 16×16 basé sur l'algorithme de Booth Radix-4, permettant de réduire le nombre de produits partiels. Dans un second temps, ce multiplieur est intégré au sein d'un accélérateur de convolution 2D utilisant un noyau de taille 3×3 , typiquement employé pour la détection de contours.

L'ensemble du système est décrit en VHDL, validé par simulation, puis synthétisé et implanté dans une technologie standard afin d'évaluer sa faisabilité matérielle.

2 Multiplieur de Booth

2.1 Principe du codage de Booth

Le codage de Booth repose sur l'observation qu'une suite de bits identiques dans un nombre binaire peut être réécrite sous forme de différence de puissances de deux. Cette transformation permet de réduire le nombre d'additions nécessaires lors de la multiplication.

Dans le cas du codage Radix-4, le multiplicateur est analysé par groupes de trois bits chevauchants $(a_{2i+1}, a_{2i}, a_{2i-1})$. Chaque triplet est associé à un coefficient appartenant à l'ensemble $\{-2, -1, 0, +1, +2\}$.

2.2 Table d'encodage Booth Radix-4

La table suivante résume le codage utilisé dans ce projet :

a_{2i+1}	a_{2i}	a_{2i-1}	Produit partiel P_i
0	0	0	0
0	0	1	+B
0	1	0	+B
0	1	1	+2B
1	0	0	-2B
1	0	1	-B
1	1	0	-B
1	1	1	0

Chaque produit partiel est ensuite décalé de $2i$ bits avant d'être additionné aux autres.

2.3 Architecture générale du multiplieur

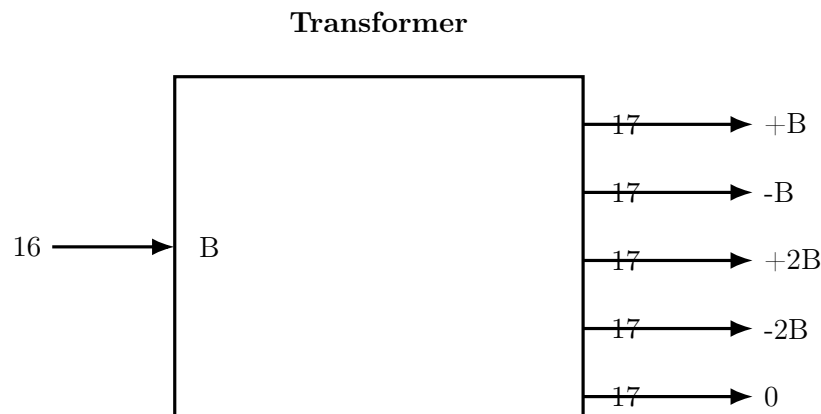
L'architecture retenue pour le multiplieur de Booth est illustrée figure ???. Elle est composée des blocs suivants :

2.3.1 Bloc Transformer

Le bloc **Transformer** prend en entrée le multiplicande B sur 16 bits et génère les multiples nécessaires au calcul des produits partiels pour le codage Booth Radix-4. Il calcule ainsi :

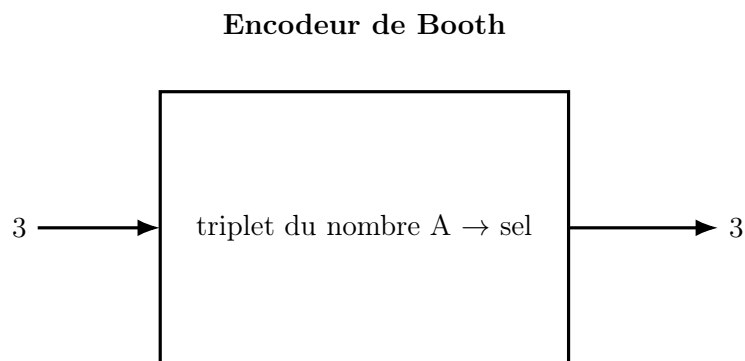
$$B, \quad 2B, \quad -B, \quad -2B, \quad 0$$

et fournit ces valeurs sur des sorties de 17 bits, afin de tenir compte des éventuels dépassements lors du décalage et de la multiplication.



2.3.2 un encodeur Booth Radix-4

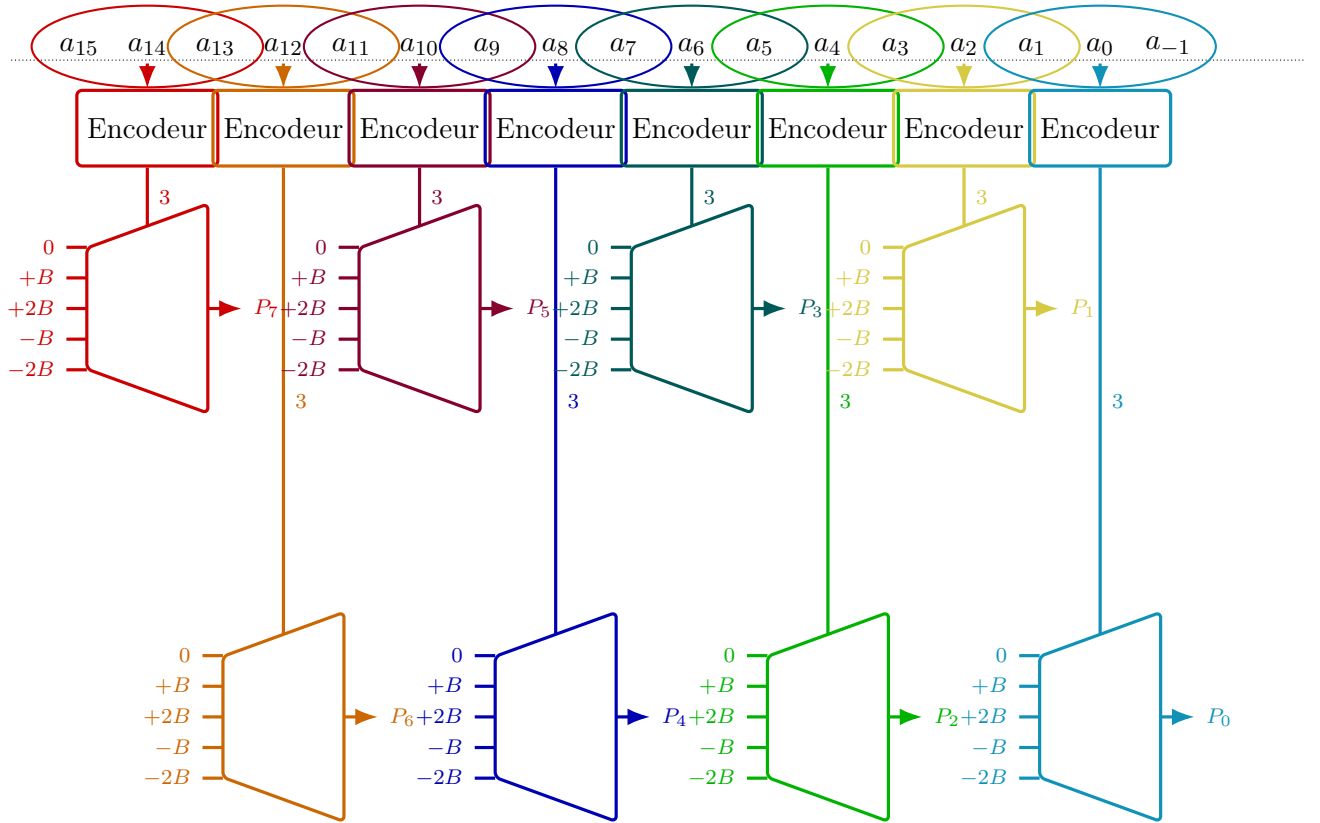
L'encodeur Booth Radix-4 analyse chaque triplet de bits du multiplicateur et génère un signal de sélection **sel**. Ce signal encode l'opération à appliquer au multiplicande, conformément au tableau de codage Booth Radix-4.



2.3.3 générateur de produits partiels basé sur un multiplexeur

Les différentes versions du multiplicande (B , $-B$, $2B$, $-2B$) sont pré-calculées et appliquées à un multiplexeur. Le signal **sel** issu de l'encodeur Booth permet de sélectionner le produit partiel correspondant, généré grâce à un multiplexeur à cinq entrées.

L'ensemble des encodeurs et des multiplexeurs sont comme suit :



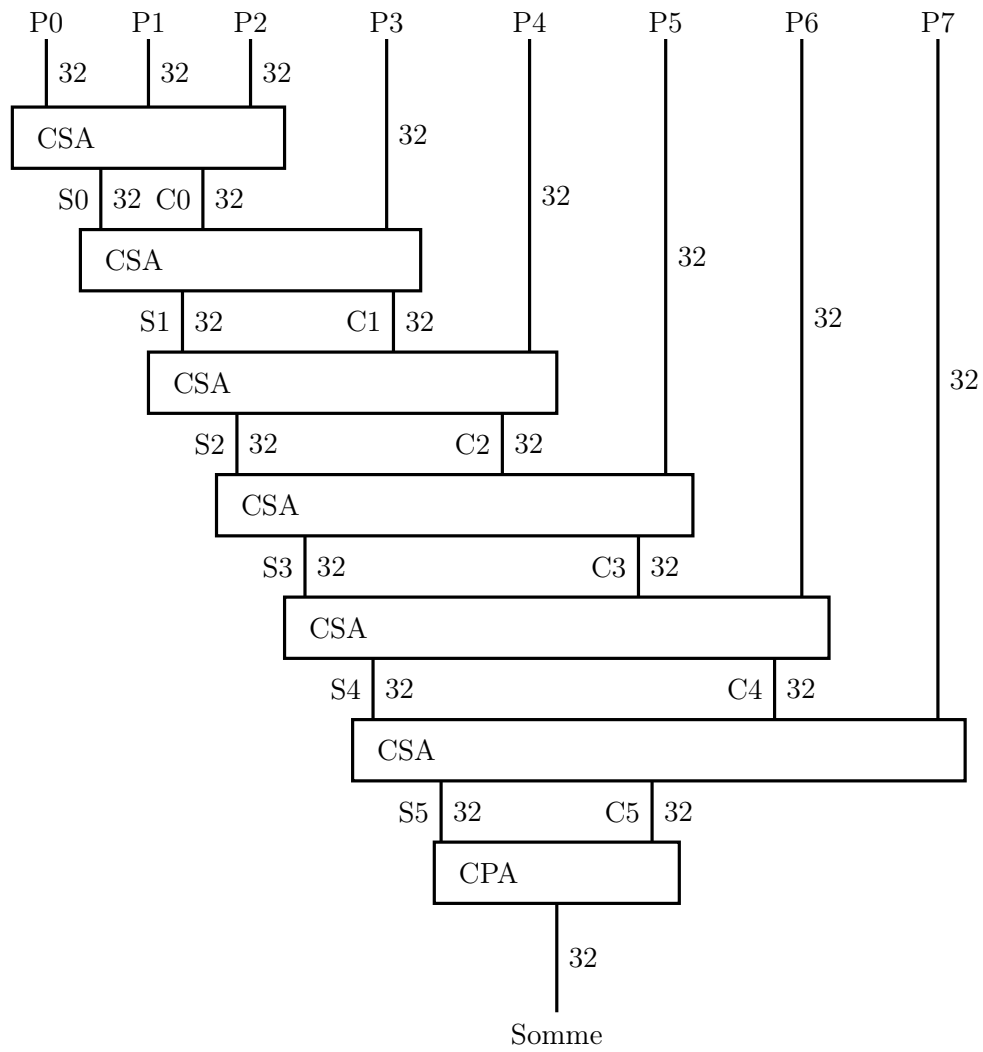
2.3.4 un arbre d'addition de type CSA (Carry Save Adder)

Les produits partiels sont additionnés à l'aide d'un arbre de Carry Save Adders (CSA). Chaque CSA additionne trois opérandes et produit deux sorties : une somme S et un vecteur de retenues C . Le résultat est interprété comme :

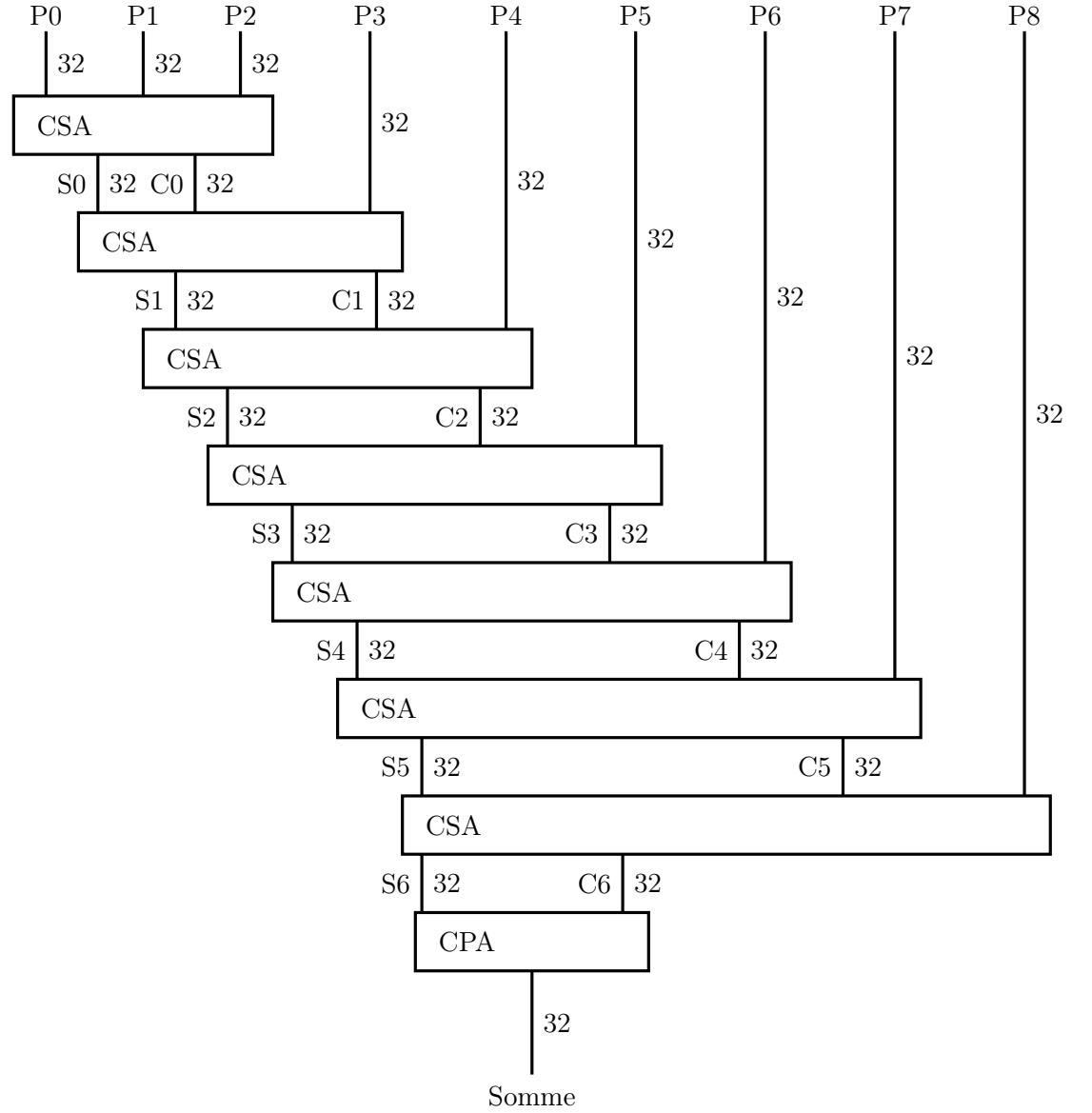
$$S + 2 \cdot C$$

Les retenues sont décalées d'un bit vers la gauche avant d'être réinjectées dans l'étage suivant. Cette approche évite la propagation immédiate des retenues et permet de réduire la profondeur critique de l'addition. Le multiplieur de Booth conçu fournit huit produits partiels (P_0-P_7), alors que le bloc de convolution en génère neuf (P_0-P_8).

Addition des 8 Produits Partiels



Addition des 9 éléments d'un produit de convolution dans le cas de notre application



2.3.5 un additionneur final CPA (Carry Propagate Adder)

Une fois l'arbre CSA réduit à deux vecteurs, un additionneur de type Carry Propagate Adder (CPA) est utilisé pour produire le résultat final. Cette étape effectue la propagation complète des retenues et fournit le produit signé sur 32 bits.

3 Convolution 2D

3.1 Principe de la convolution

La convolution 2D consiste à calculer, pour chaque pixel de l'image, une combinaison linéaire des pixels voisins pondérés par un noyau (ou kernel). Pour un noyau 3×3 , chaque sortie nécessite 9 multiplications et 8 additions.

Le principe général d'une convolution 2D est illustré à la figure 11.

Soit une fenêtre de pixels p_{ij} et un noyau de convolution composé des coefficients k_0 à k_8 . Le premier élément de sortie q_{00} est obtenu par la relation suivante :

$$\begin{aligned} q_{00} = & p_{00}k_0 + p_{10}k_1 + p_{20}k_2 \\ & + p_{01}k_3 + p_{11}k_4 + p_{21}k_5 \\ & + p_{02}k_6 + p_{12}k_7 + p_{22}k_8 \end{aligned} \quad (1)$$

Le calcul du pixel suivant dans la direction horizontale, noté q_{10} , s'effectue de manière analogue en décalant la fenêtre de convolution d'un pixel :

$$\begin{aligned} q_{10} = & p_{10}k_0 + p_{20}k_1 + p_{30}k_2 \\ & + p_{11}k_3 + p_{21}k_4 + p_{31}k_5 \\ & + p_{12}k_6 + p_{22}k_7 + p_{32}k_8 \end{aligned} \quad (2)$$

Ce mécanisme se répète ensuite pour l'ensemble de l'image, la fenêtre de convolution étant déplacée pixel par pixel afin de produire la matrice de sortie complète.

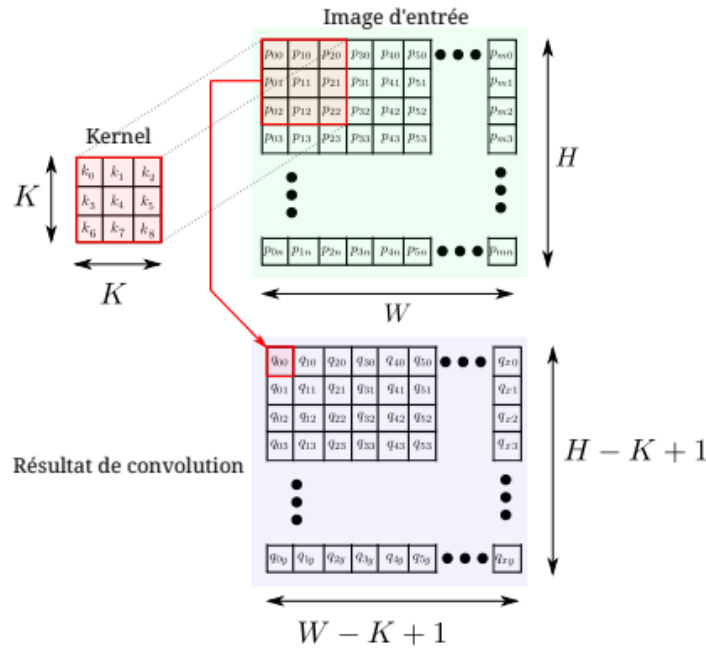


FIGURE 1 – Principe général de la convolution 2D avec un noyau 3×3

3.2 Noyaux de Sobel

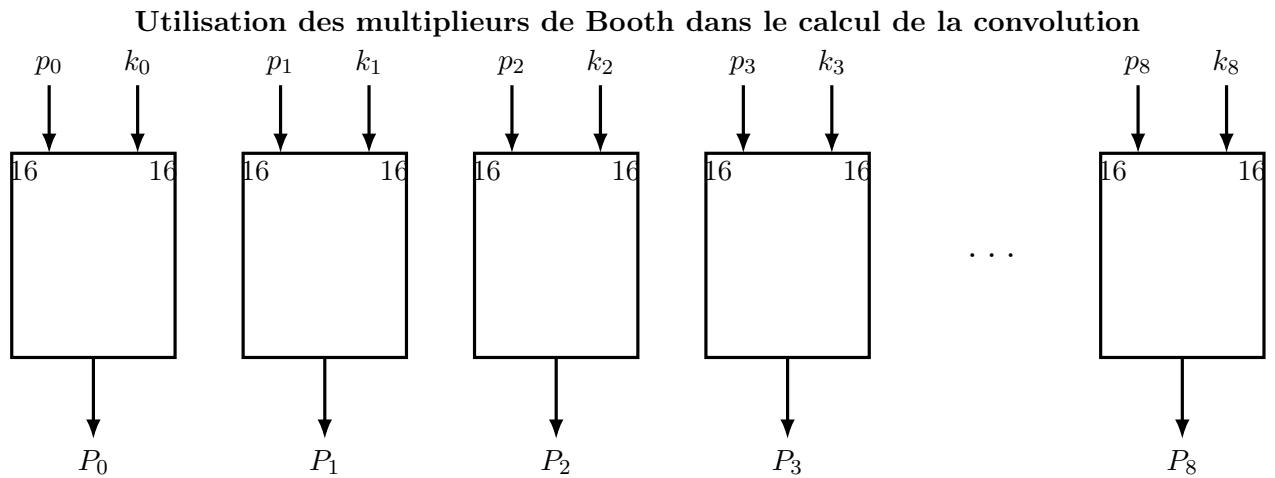
Dans ce projet, les noyaux de Sobel horizontaux et verticaux sont utilisés pour la détection de contours :

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

3.3 Cœur de convolution

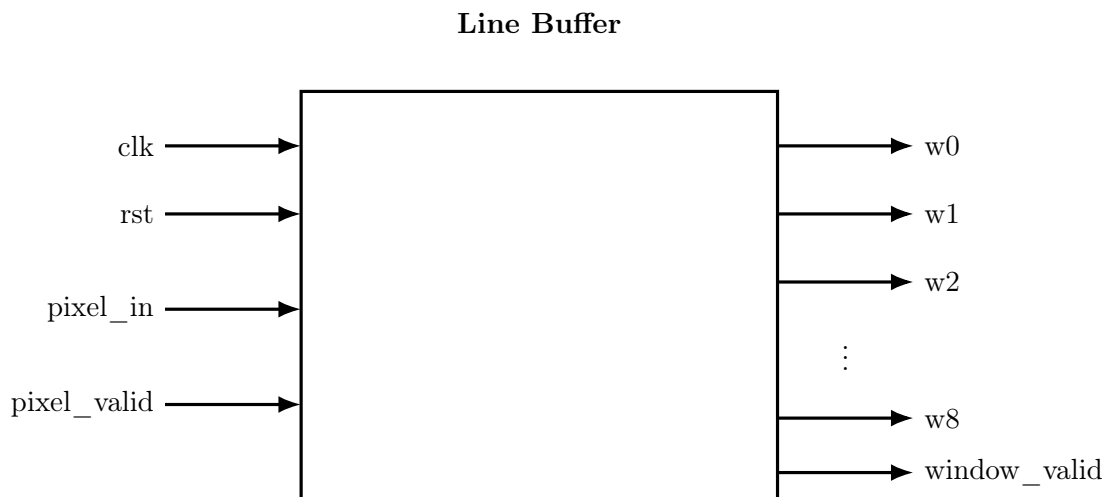
Avant de poursuivre avec la conception du cœur de convolution, nous nous sommes assurés de valider le multiplieur de Booth. Le cœur de convolution instancie neuf multi-

plieurs de Booth, suivis d'un arbre CSA adapté pour sommer les neuf produits. Le calcul est entièrement combinatoire, ce qui permet d'obtenir un débit d'un pixel par cycle après le remplissage initial du buffer.

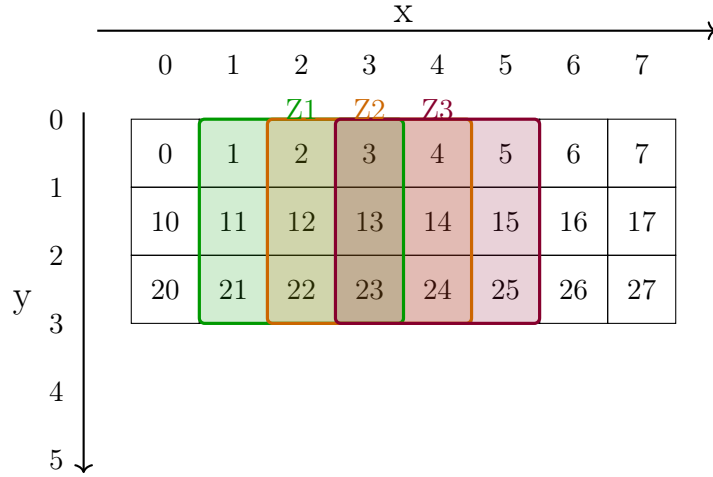


3.4 Module de stockage et de génération de fenêtre : Line Buffer

La convolution 2D nécessite, à chaque instant, l'accès simultané à un voisinage local de pixels autour du pixel courant. Dans le cas d'un noyau de convolution 3×3 , cela implique la disponibilité de neuf pixels appartenant à trois lignes consécutives de l'image. Afin de satisfaire cette contrainte tout en respectant un traitement *streaming* pixel par pixel, un module de type *line buffer* est utilisé.



Le module *line buffer* implémenté dans ce projet permet de transformer un flux séquentiel de pixels d'entrée en une fenêtre glissante 3×3 . Nous avons décidé de déplacer le noyau avec un **stride de 1** horizontalement et verticalement, comme illustré ci-dessous :



Le module repose sur deux mémoires ligne internes, représentant les deux lignes précédemment acquises de l'image, tandis que la troisième ligne correspond aux pixels reçus en temps réel.

À chaque cycle d'horloge valide, le pixel d'entrée est :

- écrit dans la première mémoire ligne,
- l'ancienne valeur de cette mémoire est propagée vers la seconde mémoire,
- et simultanément utilisé pour alimenter la ligne inférieure de la fenêtre.

Un ensemble de registres assure le décalage horizontal des pixels afin de constituer les trois colonnes successives de la fenêtre. Ainsi, à chaque position valide, les neuf sorties du module correspondent exactement aux pixels :

$$\begin{bmatrix} w_{00} & w_{01} & w_{02} \\ w_{10} & w_{11} & w_{12} \\ w_{20} & w_{21} & w_{22} \end{bmatrix}$$

où w_{11} représente le pixel central actuellement traité.

Les compteurs internes de position horizontale et verticale permettent d'identifier les zones de l'image pour lesquelles la fenêtre est pleinement définie. Un signal de validation (`window_valid`) est alors généré uniquement lorsque les trois premières lignes et les deux premières colonnes ont été correctement remplies. Ce mécanisme évite toute utilisation de données non valides en bordure d'image.

Pour une image dont la dimension en largeur est, par exemple, de 8 pixels, nous remplissons le buffer avec les trois premières lignes de l'image, puis, immédiatement après, s'alignent les trois lignes suivantes. Étant donné que le stride est de 1, le groupe de trois lignes suivant commence à partir de la deuxième ligne du groupe précédent. À chaque étape, une nouvelle ligne est ajoutée dans le buffer, et ainsi de suite.

Il s'avère que lorsque nous déplaçons notre noyau horizontalement sur l'ensemble des 8 pixels, deux groupes de 9 pixels se retrouvent redondants, comme illustré sur la figure ci-dessous par les fenêtres en bleu. Ces fenêtres constituent des zones non valides pour le calcul de la convolution.

	X																							
	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	1	2	3				
Y	0	1	2	3	4	5	6	7	10	11	12	13	14	15	16	17	20	21	22	23				
	10	11	12	13	14	15	16	17	20	21	22	23	24	25	26	27	30	31	32	33				
	20	21	22	23	24	25	26	27	30	31	32	33	34	35	36	37	40	41	42	43				

4 Module top : Chaîne complète de convolution 2D

Le module *top* constitue le niveau supérieur de l'architecture de traitement d'image développée dans ce projet. Il assure l'intégration fonctionnelle de l'ensemble des blocs nécessaires à la réalisation d'une convolution 2D 3×3 sur un flux de pixels en entrée.

Ce module reçoit en entrée :

- un flux séquentiel de pixels correspondant à une image,
- un noyau de convolution 3×3 codé en arithmétique signée.

En sortie, le module fournit un flux de pixels correspondant à l'image filtrée, ainsi qu'un signal de validation indiquant la disponibilité d'un résultat exploitable.

4.1 Architecture fonctionnelle

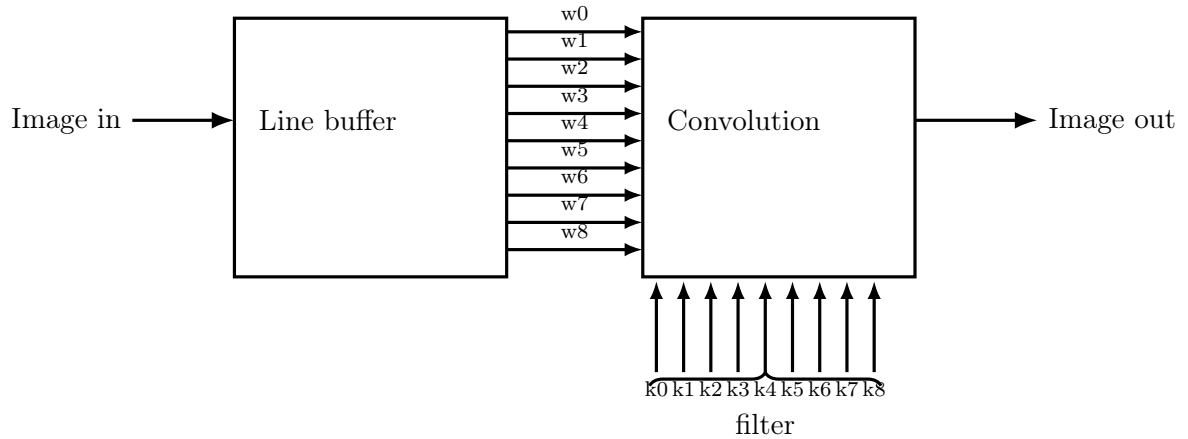
L'architecture du module top repose sur une organisation pipeline composée des blocs suivants :

- un module *line buffer*, chargé de transformer le flux d'entrée en fenêtres glissantes 3×3 et d'indiquer la validité de la fenêtre fournie
- un bloc de convolution réalisant le calcul pondéré entre la fenêtre courante et le kernel,

Le *line buffer* permet de fournir, à chaque cycle valide, les neuf pixels nécessaires au calcul de la convolution. Ces pixels sont directement transmis au bloc de convolution sans nécessiter de conversion de type, l'ensemble de l'architecture étant conçu en arithmétique signée.

Le bloc de convolution effectue alors les multiplications entre chaque pixel de la fenêtre et le coefficient correspondant du noyau, suivies d'une accumulation pour produire un pixel de sortie sur 32 bits signés. Le signal de validation est propagé afin de garantir que seuls les résultats issus de fenêtres valides soient pris en compte.

Illustration simpliste du Top Module



5 Validation du multiplieur

Le multiplieur de Booth a été validé indépendamment à l'aide d'un banc de test exhaustif couvrant :

- des opérandes positifs,
- des opérandes négatifs,
- un mix d'opérandes positifs et d'opérandes négatifs .

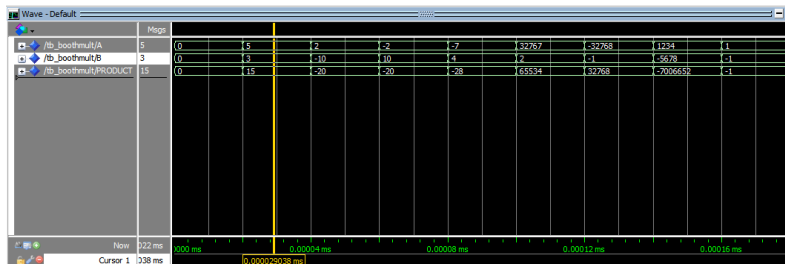


FIGURE 2 – Resultat de simulation du test du multiplieur

Les résultats obtenus correspondent systématiquement au produit arithmétique attendu en complément à deux.

6 Validation fonctionnelle de la convolution

La validation fonctionnelle de l'architecture a été réalisée à l'aide d'un testbench dédié, basé sur une image de petite taille permettant une vérification analytique précise des résultats obtenus.

6.1 Image d'entrée

L'image de test utilisée est une image de dimension 4×8 , définie comme suit :

$$I = \begin{bmatrix} 0 & 10 & 20 & 30 & 40 & 0 & 10 & 20 \\ 0 & 10 & 20 & 30 & 40 & 0 & 10 & 20 \\ 0 & 10 & 20 & 30 & 40 & 0 & 10 & 20 \\ 0 & 10 & 20 & 30 & 40 & 0 & 10 & 20 \end{bmatrix}$$

6.2 Noyau de convolution utilisé

Le noyau de Sobel horizontal utilisé pour la convolution est le suivant :

$$K = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

Ce noyau permet de mettre en évidence les variations d'intensité selon l'axe horizontal de l'image, c'est-à-dire les contours verticaux.

6.3 Calcul de la convolution : exemples détaillés

La convolution ne peut être calculée que pour les positions où la fenêtre 3×3 est entièrement définie. Dans le cas de l'image 4×8 , cela correspond aux lignes 1 à 2 et aux colonnes 1 à 6, ce qui conduit à une image de sortie de dimension 2×6 .

6.3.1 Exemple de calcul 1

Considérons un pixel centré en position (ligne 1, colonne 2). La fenêtre associée est :

$$\begin{bmatrix} 10 & 20 & 30 \\ 10 & 20 & 30 \\ 10 & 20 & 30 \end{bmatrix}$$

Le calcul de la convolution s'effectue comme suit :

$$\begin{aligned} q &= (-1 \cdot 10 + 0 \cdot 20 + 1 \cdot 30) \\ &\quad + (-2 \cdot 10 + 0 \cdot 20 + 2 \cdot 30) \\ &\quad + (-1 \cdot 10 + 0 \cdot 20 + 1 \cdot 30) \\ &= 20 + 40 + 20 = 80 \end{aligned}$$

6.3.2 Exemple de calcul 2

$$\begin{bmatrix} 30 & 40 & 0 \\ 30 & 40 & 0 \\ 30 & 40 & 0 \end{bmatrix}$$

Le calcul de la convolution donne :

$$\begin{aligned} q &= (-1 \cdot 30 + 0 \cdot 40 + 1 \cdot 0) \\ &\quad + (-2 \cdot 30 + 0 \cdot 40 + 2 \cdot 0) \\ &\quad + (-1 \cdot 30 + 0 \cdot 40 + 1 \cdot 0) \\ &= -30 - 60 - 30 = -120 \end{aligned}$$

6.4 Résultat final de la convolution

L'image de sortie obtenue après convolution est donc :

$$\begin{bmatrix} 80 & 80 & 80 & -120 & -120 & 80 \\ 80 & 80 & 80 & -120 & -120 & 80 \end{bmatrix}$$

Ce résultat obtenu par analyse théorique est cohérent avec les résultats de la simulation présentés ci-dessous.

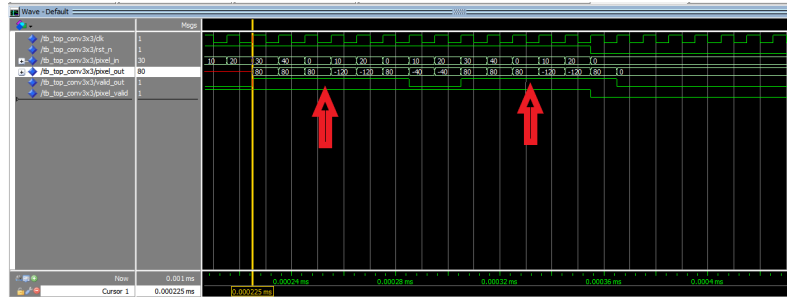


FIGURE 3 – Resultat de simulation du module complet avec le filtre sobel horizontal

6.5 Filtre de Sobel vertical

En complément du filtre de Sobel horizontal, nous avons également validé le fonctionnement de l'architecture avec un filtre de Sobel vertical. Ce filtre est destiné à mettre en évidence les variations d'intensité selon l'axe vertical de l'image, c'est-à-dire les contours horizontaux.

6.5.1 Noyau de Sobel vertical

Le noyau de Sobel vertical utilisé est défini comme suit :

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Contrairement au filtre horizontal, ce noyau compare les lignes supérieures et inférieures de la fenêtre 3×3 , ce qui permet de détecter les transitions d'intensité verticales dans l'image.

6.5.2 Analyse théorique sur l'image de test

L'image de test utilisée précédemment est rappelée ci-dessous :

$$I = \begin{bmatrix} 0 & 10 & 20 & 30 & 40 & 0 & 10 & 20 \\ 0 & 10 & 20 & 30 & 40 & 0 & 10 & 20 \\ 0 & 10 & 20 & 30 & 40 & 0 & 10 & 20 \\ 0 & 10 & 20 & 30 & 40 & 0 & 10 & 20 \end{bmatrix}$$

Toutes les lignes de cette image étant identiques, il n'existe aucune variation d'intensité selon l'axe vertical. Théoriquement, l'application du filtre de Sobel vertical sur cette image doit donc produire des valeurs nulles ou très proches de zéro, en dehors des effets de bord.

6.5.3 Exemple de calcul

Considérons une fenêtre 3×3 centrée sur un pixel valide, par exemple :

$$\begin{bmatrix} 10 & 20 & 30 \\ 10 & 20 & 30 \\ 10 & 20 & 30 \end{bmatrix}$$

Le calcul de la convolution avec le noyau G_y donne :

$$\begin{aligned} q &= (-1 \cdot 10 + -2 \cdot 20 + -1 \cdot 30) \\ &\quad + (0 \cdot 10 + 0 \cdot 20 + 0 \cdot 30) \\ &\quad + (1 \cdot 10 + 2 \cdot 20 + 1 \cdot 30) \\ &= (-10 - 40 - 30) + 0 + (10 + 40 + 30) \\ &= 0 \end{aligned}$$

Ce résultat confirme l'absence de gradient vertical dans l'image.

6.5.4 Résultat global

Pour l'ensemble des positions valides de la convolution, le résultat obtenu avec le filtre de Sobel vertical est une image de sortie composée uniquement de valeurs nulles :

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Ce comportement est conforme aux attentes théoriques et valide la bonne implémentation du filtre de Sobel vertical dans l'architecture matérielle. Il démontre également que le module de convolution est capable de traiter correctement différents noyaux sans modification structurelle. La validation par simulation est présentée dans l'image ci-dessous.

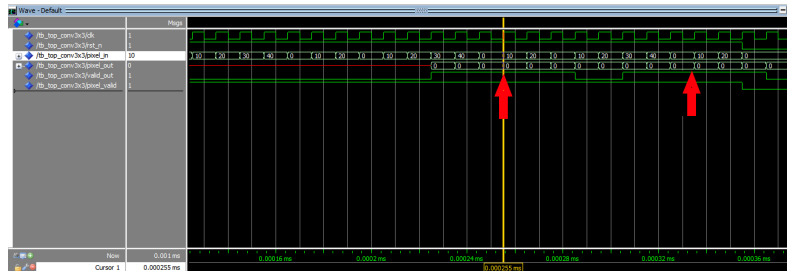


FIGURE 4 – Resultat de simulation du module complet avec le filtre sobel vertical

7 Synthèse et placement-routage

L'ensemble du système a été synthétisé dans la technologie cible GPDK 45,nm de Cadence. Les résultats de la synthèse indiquent que le circuit peut fonctionner avec une période maximale de $2.45\mu s$, ce qui correspond à une fréquence d'environ 408,kHz. La surface totale occupée par le circuit, pour un fonctionnement à cette fréquence, est de $0.00573\mu m^2$. Enfin, la puissance consommée est estimée à $0.115\mu W$. Les étapes de placement et de routage ont permis de vérifier la faisabilité physique de l'architecture.

Les résultats issus de la synthèse et placement-routage les contraintes temporelles sont respectées.

Pour la synthèse il à été important de s'assurer que le code écrit est synthétisable et que le chargement des modules du design se fassent dans un ordre spécifique (les blocs élémentaire en première position puis ensuite les blocs complexes conçus à partir des blocs élémentaires).

Les résultats de simulation post-synthèse et post placement routage nous ont permis de faire la validation finale de l'accélérateur.

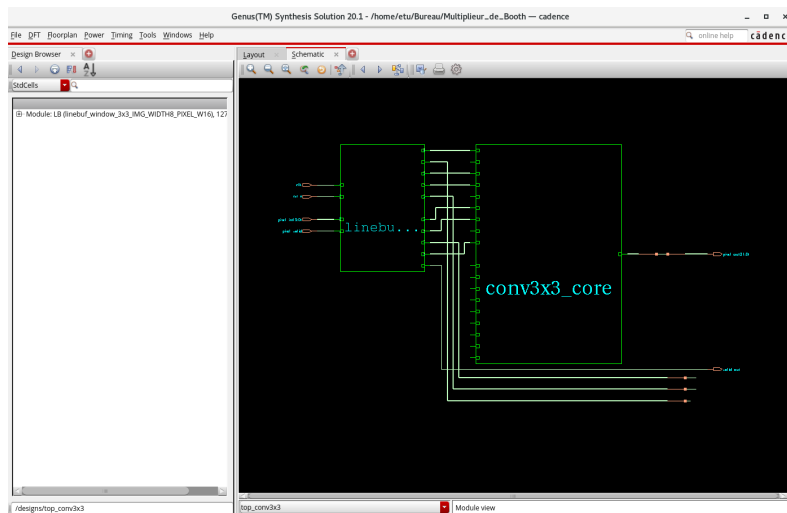


FIGURE 5 – Schema de l'accélérateur après synthèse

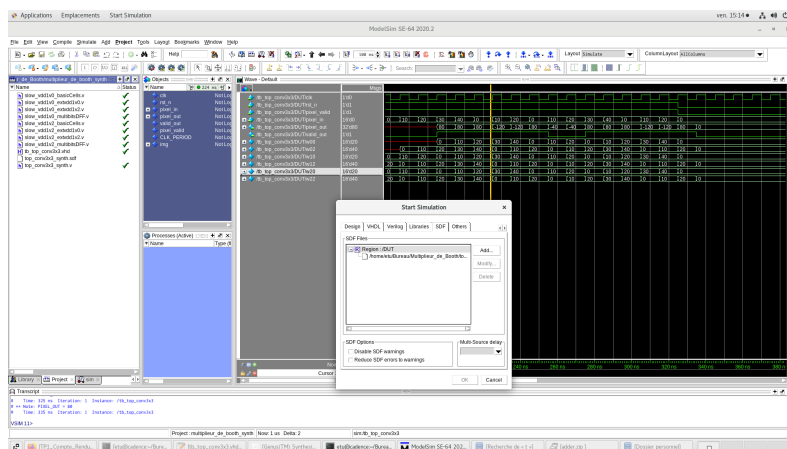
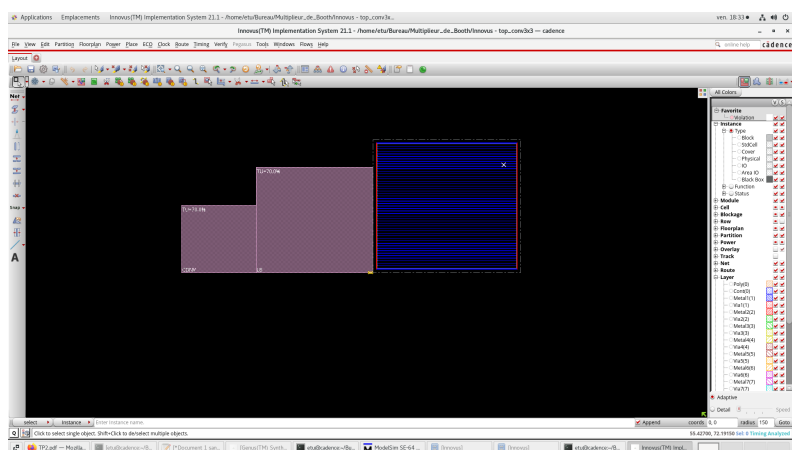
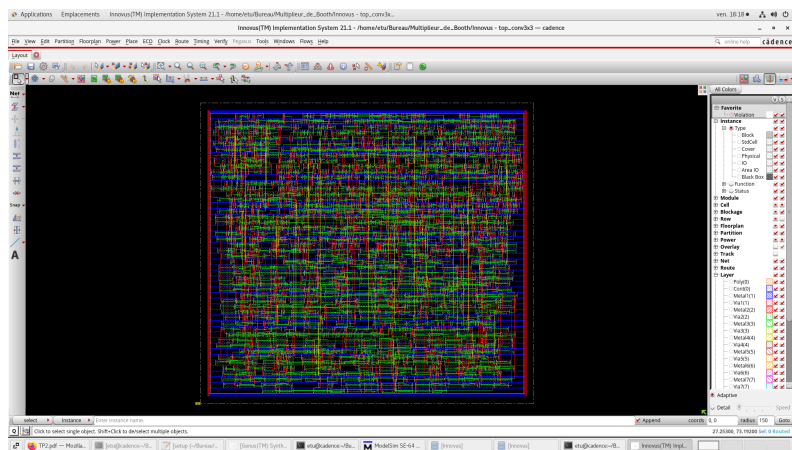
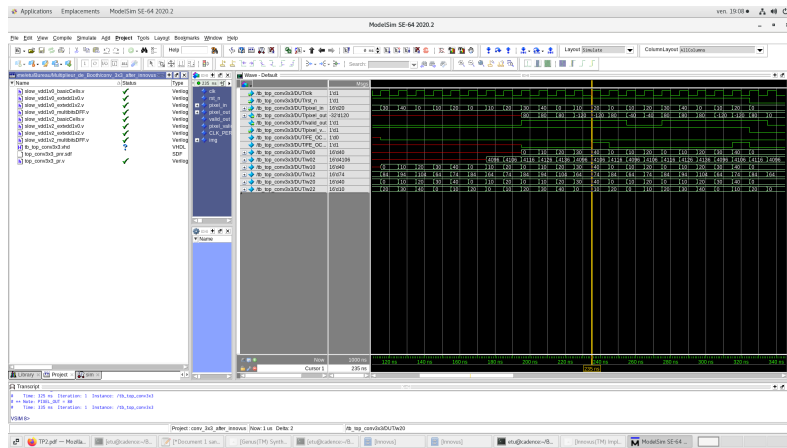


FIGURE 6 – Resultat de simulation du module après la synthèse



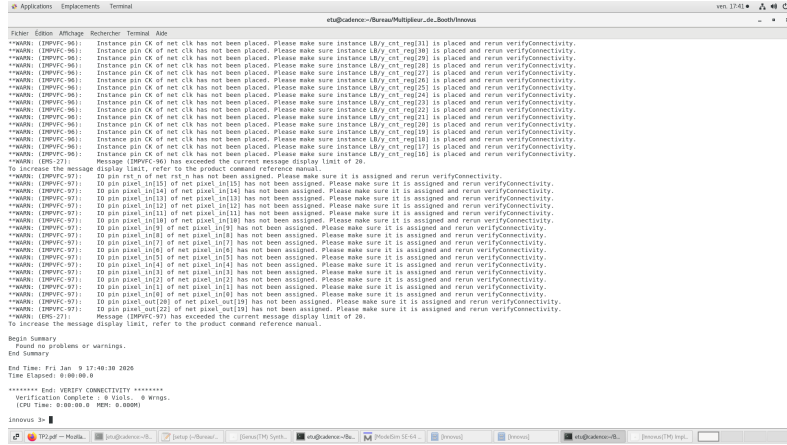


FIGURE 10 – Connectivity report

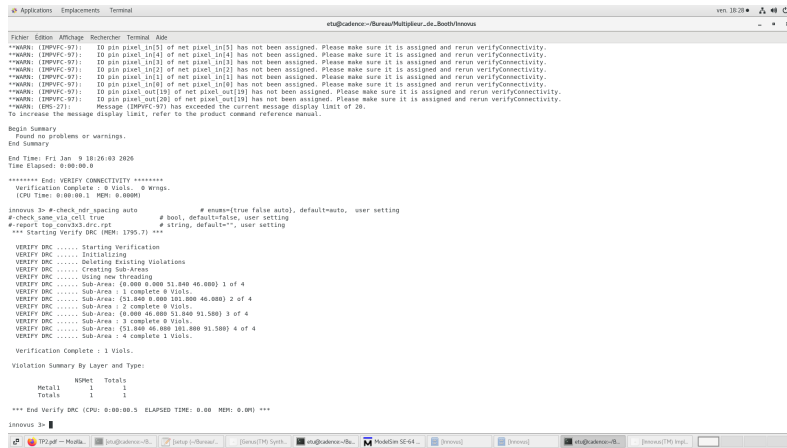


FIGURE 11 – DRC Report

8 Conclusion

Ce projet a permis la conception et la validation d'un multiplieur signé Booth Radix-4 de dimension 16×16 , ainsi que son intégration efficace au sein d'un accélérateur de convolution 2D. Une partie significative du flot de conception ASIC a pu être menée à bien, en s'appuyant sur une architecture adaptée aux contraintes fonctionnelles et technologiques imposées par le projet.

Lors de la phase de conception physique sous Cadence Innovus, une unique violation des règles de conception (DRC) a été détectée lors de la vérification finale du layout. Malgré une analyse détaillée du rapport DRC, cette erreur n'a pas pu être corrigée dans le temps imparti, ce qui a empêché la poursuite de certaines étapes finales du flot de conception.

Cette violation est localisée dans une zone spécifique du design et n'affecte ni le bon fonctionnement logique du circuit ni les résultats obtenus lors des phases de synthèse, de placement et de routage. Elle met néanmoins en évidence certaines contraintes liées aux règles technologiques de la bibliothèque utilisée ou aux paramètres du flot automatique d'Innovus.

Dans un contexte industriel, cette violation aurait pu être résolue par une itération supplémentaire sur les contraintes de routage, un ajustement des règles de conception ou une intervention manuelle au niveau du layout.

Par ailleurs, plusieurs perspectives d'amélioration du système peuvent être envisagées. Il serait notamment pertinent d'intégrer un module de sélection dynamique des noyaux de convolution, plutôt que de les définir comme des constantes. De plus, rendre l'architecture plus flexible, en permettant la prise en charge d'un stride paramétrable ainsi que de tailles de noyaux arbitraires, constituerait une évolution intéressante du système.

Références

- **TP Projet – Master 2 EEA, parcours EMB** : Conception d'un multiplieur de Booth 16×16 et son intégration dans un accélérateur de convolution 2D.
- **MIT – Online FPGA Resources** : Convolution 2D sur FPGA. <https://fpga.mit.edu/6205/F25/assignments/week07/convolution>